
Prasanna Sanjay Kasar

Email: prasanna.kasar06@gmail.com

GSoC 2025 Exercises for SOFIE Projects: GPU Support for Machine Learning Inference

20th March 2025

Table of Contents

1. **Overview**
2. **Preliminary Tasks Completed**
3. **Exercise 3 – Play with the SOFIE SYCL Implementation**
4. **Exercise 4 – Implement ONNX Operators in GPU**
 - 4.1 Objective
 - 4.2 Approach
 - 4.2.1 Initial Setup
 - 4.2.2 Function Design
 - 4.2.3 Tanh Implementation
 - 4.2.4 Testing
 - 4.2.5 Unit Test and Integration
 - 4.3 Outcome
5. **Future Work**
6. **Conclusion**

Overview

This document outlines the steps undertaken to complete the exercises for the Google Summer of Code (GSoC) 2025 project titled "TMVA SOFIE - GPU Support for Machine Learning Inference." The primary objective of the project is to improve GPU support for machine learning inference within the TMVA (Toolkit for Multivariate Analysis) framework, specifically in its SOFIE implementation. The exercises involved working with SYCL and OpenCL-based GPU inference for ONNX operators, along with implementing and testing these operators.

1. **Personal Development Branch:**

GitHub - PrasannaKasar Root Branch:

<https://github.com/PrasannaKasar/root/tree/prasanna>

2. Exercise 4 Solutions:

GitHub - TMVA SOFIE Exercise 4 Solutions:

<https://github.com/PrasannaKasar/root/tree/prasanna/tmva/sofie>

3. Preliminary Tasks:

https://docs.google.com/document/d/1TZWTqdAzkDGCTOEmeXiAMyxhBc5nM4bt8u_FHRi-gmY/edit?usp=sharing

Preliminary Tasks Completed

Before starting the exercises, I completed the following preliminary tasks as described in the project document:

- Familiarized myself with the ROOT and TMVA frameworks.
- Set up the development environment, including installing the required dependencies and ensuring that the ROOT build system worked smoothly.
- Completed initial exercises and reviewed the available documentation, including the parsing algorithm and ONNX operator examples.

Exercise 3 – Play with the SOFIE SYCL Implementation

For Exercise 3, I explored the SYCL implementation of the SOFIE framework, which is designed to leverage GPUs for machine learning inference. I thoroughly examined the existing architecture, focusing on how it manages GPU computation, kernel execution, and memory management. The SYCL implementation served as a strong foundation for my subsequent work on the project, particularly when implementing new operators on GPUs using OpenCL.

Key Observations:

- The SYCL-based implementation defines a **GeneratedGPU** function, which explicitly handles GPU context creation, kernel compilation, and invocation.
- The existing structure for managing kernel code and memory allocation was modular, providing a blueprint for how to implement additional operators.

Exercise 4 – Implement ONNX Operators in GPU

Objective

The objective of this exercise was to implement an ONNX operator on a GPU using a stack of choice, such as CUDA, ROCm, or ALPAKA. I chose to implement the **Tanh** operator using **OpenCL**. The implementation was inspired by the SYCL-based GPU code for SOFIE and aimed to integrate a new GPU-based ONNX operator into the existing framework.

Approach

1. Initial Setup:

- I began by analyzing the existing SYCL implementation to understand how operators were being implemented for GPU.
- I found that the **GeneratedGPU** function, present in **RModel1.cxx**, played a central role in compiling OpenCL code, handling the GPU context, and managing buffers.
- The functions implements 'infer' functions which take in input tensor, perform required operation on it and return target tensor.

```

void RModel::GenerateGPUOpenCL(std::underlying_type_t<Options> options, int batchSize, long pos, bool verbose) {
    std::string hgname = fName;
    std::transform(hgname.begin(), hgname.end(), hgname.begin(), [](unsigned char c) {
        return std::toupper(c);
    });
    fGC += "#ifndef ROOT_TMVA_SOFIE_" + hgname + "_GPU \n";
    fGC += "#define ROOT_TMVA_SOFIE_" + hgname + "_GPU \n";
    fGC += "\n";
    fGC += "#include <cmath> \n";
    fGC += "#include <vector> \n";
    fGC += "#include \"TMVA/SOFIE_common.hxx\" \n";
    fGC += "#include <CL/cl.h>\n";
    fGC += "#include <stdio.h>\n";
    fGC += "#include <stdlib.h>\n";
    fGC += "#include <math.h>\n";
    fGC += "\n";
    fGC += "namespace TMVA_SOFIE_" + fName + "_GPU { \n";
    fGC += "struct Session { \n";
    fGC += "\n";
    fGC += "// kernel source function \n";
    fGC += "\n";
    f0Operators[0]->Initialize(*this);
    fGC += (f0Operators[0]->GenerateGPUOpenCL(std::to_string(0)));
    // fGC += (f0Operators[i]->Generate(std::to_string(i)));
    std::string length = f0Operators[0]->GetLength();
    fGC += ("#define VECTOR_SIZE " + length);
    fGC += "\n";
    fGC += "char* fIntermediateMemoryPool = new char[" + length + "*4" + "]; \n";
    fGC += "float* C = reinterpret_cast<float*>(fIntermediateMemoryPool + 0); \n";
    fGC += "\n";
    fGC += "Session(std::string = \"\") { \n";
    fGC += "    //---- allocate the intermediate dynamic tensors \n";
    fGC += "}; \n";
    fGC += "std::vector<float> infer(float* tensor_input) { \n";
    fGC += "    float* A = tensor_input; \n";
    fGC += "\n";
    fGC += "    cl_int err;\n";
    fGC += "    cl_uint numPlatforms;\n";
    fGC += "    err = clGetPlatformIDs(0, NULL, &numPlatforms);\n";
    fGC += "    if (err != CL_SUCCESS || numPlatforms == 0) {\n";
    fGC += "        fprintf(stderr, \"Error: Failed to find any OpenCL platforms. Error code %d\\n\", err);\n";
    fGC += "        return std::vector<float> {};\n";
    fGC += "    }\n";
    fGC += "    cl_platform_id *platforms = (cl_platform_id *)malloc(numPlatforms * sizeof(cl_platform_id));\n";
}

```

(fig 4.1 First 50 lines of GenerateGPUOpenCL function in RModel.cxx)

2. Function Design:

- I created a similar structure for implementing the **Tanh** operator:
 - I defined two methods: **GeneratedGPU** and **GetLength**, both of which were declared in **ROperator.hxx** as **virtual** functions. By default, these methods return an empty string, as they were not implemented for every operator initially.
 - The **GeneratedGPU** function is responsible for generating the OpenCL kernel code, setting up the OpenCL context, and launching the kernel on the GPU.
 - The **GetLength** function returns the size of the tensors to allocate the appropriate memory.

```
virtual std::string GenerateGPUOpenCL(std::string OpName) {return ""};
virtual std::string GetLength() {return ""};
```

(fig 4.2 Adding virtual functions to ROperator.hxx)

3. Tanh Implementation:

- I then created a specialized class for the **Tanh** operator in `ROperator_tanh.hxx`, overriding the `GeneratedGPU` and `GetLength` functions.
 - The `GeneratedGPU` function generates the OpenCL code specific to the **Tanh** operator.
 - The `GetLength` function determines the size of the input tensors for memory allocation.
- The `RModel.cxx` file was modified to ensure that the generated OpenCL code was correctly compiled and executed.

```
std::string GetLength() override {
    size_t length = ConvertShapeToLength(fShape);
    return std::to_string(length);
}

std::string GenerateGPUOpenCL(std::string OpName) override {
    OpName = "Tanh";

    if (fShape.empty()) {
        throw std::runtime_error("OpenCL error: TMVA SOFIE Operator Tanh called to Generate without being initialized first");
    }

    std::stringstream out;
    size_t length = ConvertShapeToLength(fShape);

    out << "\n//----- TANH\n";
    out << "const char* kernelSource = \"__kernel void \" << OpName
    out << \"(_global const float* A, __global float* C) {\n\n\";
    out << \"    int id = get_global_id(0);\n\n\";
    out << \"    if (id < \" << length << \" ) {\n\n\";
    out << \"        C[id] = tanh(A[id]);\n\n\";
    out << \"    }\n\n\";
    out << \"}\n\n\";

    return out.str();
}
```

(fig 4.3 Writing GenerateGPUOpenCL and GetLength functions which override virtual functions in ROperator.hxx)

4. Testing:

- After implementing the **Tanh** operator, I focused on testing the GPU-based implementation.
- I copied the original test from **TestCustomModelsFromONNX** but changed the namespace to `TMVA_SOFIE_Tanh_GPU` to distinguish the GPU-specific test.
- I used a **Tanh.ONNX** model and verified that the OpenCL code was generated correctly by checking the kernel's output.

- I created a reference file `TanhGPU.hxx.ref` to compare the expected and actual outputs of the OpenCL-based **Tanh** operator.
- I included this reference file in the test suite, ensuring that the generated kernel was correct.

```
#include "/home/prasanna/RootDevelopment/root/tutorials/machine_learning/Tanh.hxx"
#include "/home/prasanna/RootDevelopment/root/tmva/sofie/test/input_models/references/TanhGPU.ref.hxx"

#include "gtest/gtest.h"

constexpr float DEFAULT_TOLERANCE = 1e-3f;

TEST(ONNX, Tanh_OpenCL)
{
    std::cout << "Testing Tanh Operator \n";

    constexpr float TOLERANCE = DEFAULT_TOLERANCE;

    // Preparing the random input
    std::vector<float> input({
        -0.3896, -0.3521, 0.0363, 1.0962, 0.5085, -0.8523, -0.6766, 0.2421,
        1.5971, 1.3873, -0.2112, -0.6895, -0.5069, -2.1395, -0.7087, 1.1658,
        1.3493, 0.8132, 1.7156, -0.8637, -0.1971, 0.0411, -0.5662, -0.2516
    });

    TMVA_SOFIE_Tanh_GPU::Session s("Tanh_FromONNX.dat");

    std::vector<float> output = s.infer(input.data());

    // Checking output size
    EXPECT_EQ(output.size(), sizeof(Tanh_ExpectedOutputGPU::outputs) / sizeof(float));

    float *correct = Tanh_ExpectedOutputGPU::outputs;

    // Checking every output value, one by one
    for (size_t i = 0; i < output.size(); ++i) {
        EXPECT_LE(std::abs(output[i] - correct[i]), TOLERANCE);
    }
}
```

(fig 4.4 Unit test to Tanh operator)

```
namespace Tanh_ExpectedOutputGPU{
    float outputs[] = {
        -0.3710, -0.3382, 0.0363, 0.7991, 0.4688, -0.6923, -0.5893, 0.2375,
        0.9212, 0.8826, -0.2081, -0.5976, -0.4675, -0.9727, -0.6099, 0.8229,
        0.8739, 0.6714, 0.9373, -0.6982, -0.1946, 0.0411, -0.5125, -0.2464
    };
} // namespace Tanh_ExpectedOutputGPU
```

(fig 4.5 Expected answer after Tanh operation)

5. Parsing Tanh ONNX model to generate equivalent OpenCL code:

- To verify whether the `GenerateGPUOpenCL` function produces equivalent OpenCL code, I parsed a Tanh ONNX model using the `RModelParser_ONNX` parser.
- Then, I called the `GenerateGPUOpenCL` and `OutputGenerated` functions, which created the `Tanh.hxx` header containing the generated OpenCL code.

```
1  /// \file
2  /// \ingroup tutorial_ml
3  /// \notebook -nodraw
4  /// This macro provides a simple example for the parsing of ONNX files into
5  /// RModel object and further generating the .hxx header files for inference.
6  ///
7  /// \macro_code
8  /// \macro_output
9
10 using namespace TMVA::Experimental;
11
12 void parse_onnx_op_tanh(std::string inputFile = ""){
13     if (inputFile.empty() )
14         inputFile = "/home/prasanna/RootDevelopment/root/tmva/sofie/test/input_models/Tanh.onnx";
15
16     // std::printf("2");
17
18     //Creating parser object to parse ONNX files
19     SOFIE::RModelParser_ONNX parser;
20     SOFIE::RModel model = parser.Parse(inputFile, true);
21
22     //Generating inference code
23     model.GenerateGPUOpenCL();
24     model.OutputGenerated();
25 }
26
```

(fig 4.6 Parsing sample Tanh ONNX operator to generate equivalent OpenCL code)

```

1  #ifndef ROOT_TMVA_SOFIE_TANH_GPU
2  #define ROOT_TMVA_SOFIE_TANH_GPU
3
4  #include <cmath>
5  #include <vector>
6  #include "TMVA/SOFIE_common.hxx"
7  #include <CL/cl.h>
8  #include <stdio.h>
9  #include <stdlib.h>
10 #include <math.h>
11
12 namespace TMVA_SOFIE_Tanh_GPU {
13 struct Session {
14
15     // kernel source function
16
17
18     //----- TANH
19     const char* kernelSource = "__kernel void Tanh(__global const float* A, __global float* C) {\n"
20     "    int id = get_global_id(0);\n"
21     "    if (id < 24) {\n"
22     "        C[id] = tanh(A[id]);\n"
23     "    }\n"
24     "};";
25     #define VECTOR_SIZE 24
26     char* fIntermediateMemoryPool = new char[24*4];
27     float* C = reinterpret_cast<float*>(fIntermediateMemoryPool + 0);
28
29     Session(std::string = "") {
30         //---- allocate the intermediate dynamic tensors
31     }
32     std::vector<float> infer(float* tensor_input) {
33         float* A = tensor_input;
34
35         cl_int err;
36         cl_uint numPlatforms;
37         err = clGetPlatformIDs(0, NULL, &numPlatforms);
38         if (err != CL_SUCCESS || numPlatforms == 0) {
39             fprintf(stderr, "Error: Failed to find any OpenCL platforms. Error code %d\n", err);
40             return std::vector<float> {};
41         }
42         cl_platform_id *platforms = (cl_platform_id *)malloc(numPlatforms * sizeof(cl_platform_id));
43         err = clGetPlatformIDs(numPlatforms, platforms, NULL);
44         if (err != CL_SUCCESS) {
45             fprintf(stderr, "Error: Failed to get OpenCL platform IDs. Error code %d\n", err);
46             free(platforms);
47             return std::vector<float> {};
48         }

```

(fig 4.7 First 50 lines of generated OpenCL code for Tanh operator)

6. OpenCL Integration:

- Since OpenCL was required for the GPU execution, I updated the `CMakeLists.txt` file in the `test` directory to include the `-lOpenCL` flag for proper compilation.


```

# Creating a Google Test
if (tmva-cpu) # we need BLAS for compiling the models
ROOT_ADD_GTEST(TestCustomModelsFromONNX TestCustomModelsFromONNX.cxx
  LIBRARIES
    ROOTTMVASofie
    -lOpenCL
    ${BLAS_LINKER_FLAGS}
    ${BLAS_LIBRARIES}
  INCLUDE_DIRS
    ${CMAKE_CURRENT_BINARY_DIR}
)

```

(fig 4.7 Adding -lOpenCL command to run tests with OpenCL code)

Outcome

- The **Tanh** operator was successfully implemented and tested on GPU using OpenCL.
- The tests were integrated into the existing test suite, and I ensured that the implementation did not interfere with CPU-based operations. The OpenCL-based test passed successfully without any errors.
- The reference files were updated to ensure that the generated results could be validated.

```

393: Running main() from /home/prasanna/RootDevelopment/
393: [=====] Running 114 tests from 1 test suite.
393: [-----] Global test environment set-up.
393: [-----] 114 tests from ONNX
393: [ RUN      ] ONNX.Tanh_OpenCL
393: Testing Tanh Operator
393: [      OK  ] ONNX.Tanh_OpenCL (286 ms)

```

```

1/1 Test #393: gtest-tmva-sofie-TestCustomModelsFromONNX ... Passed

The following tests passed:
  gtest-tmva-sofie-TestCustomModelsFromONNX

100% tests passed, 0 tests failed out of 1

```

(fig 4.7 Test case passed for generated Tanh code)

Test Results:

- The test was run on the **Tanh** function implemented in **OpenCL**. It passed successfully with no failures, demonstrating that the OpenCL implementation is correct and functional.

Future Work

- **Implement Additional ONNX Operators:**
 - I plan to extend the current work by implementing other ONNX operators, such as **ReLU**, **Elu**, and **Sigmoid**, on the GPU using OpenCL. These operators are frequently used in machine learning models and would help further improve the GPU support in the TMVA SOFIE framework.
- **Optimize GPU Performance:**
 - There is potential for optimizing the performance of GPU implementations by considering memory coalescing, kernel fusion, and other OpenCL-specific optimizations.
- **Bonus Tasks:**
 - I aim to explore additional advanced features of the TMVA SOFIE framework, potentially involving multi-GPU setups or other optimization strategies.